

Practica 9: Cálculo de eigenvalores con el método QR

El método QR se basa en el hecho de que a una matriz real A de tamaño $n \times n$ se le puede calcular la factorización QR de forma que

$$A = QR$$

Donde Q es ortogonal y R es triangular superior. El método QR se define de la siguiente manera:

- Inicialización: $A_0 = A$, obtenemos $Q_0 = Q$ y $R_0 = R$ con la factorización QR de A_0 .
- Iteración 1: $A_1 = R_0 Q_0$, después factorizar $A_1 = Q_1 R_1$
- Iteración 2: $A_2 = R_1 Q_1$, después factorizar $A_2 = Q_2 R_2$
- Iteración 3: $A_3 = R_2 Q_2$, después factorizar $A_3 = Q_3 R_3$
- $\vdots$
- Iteración m : $A_m = R_{m-1} Q_{m-1}$, obtenemos factorizar $A_m = Q_m R_m$

Conforme m crece los valores propios de A_m se aproximan a los valores propios de A .

En la práctica anterior ustedes programaron una función que calcula la descomposición QR de una matriz A y que regrese las matrices Q y R que aparecen en el paso 5 del algoritmo

Pero... ¿Por qué funciona el algoritmo?

La respuesta a esta pregunta se encuentra en el siguiente Teorema:

Teorema:

Sea A una matriz real de tamaño $n \times n$ con eigenvalores que satisfacen

$$|\lambda_1| > |\lambda_2| > \dots > |\lambda_n| > 0$$

Entonces las matrices A_k en el método QR convergen a una matriz triangular superior con entradas en la diagonal $\{\lambda_m\}$, $m=1, 2, \dots, n$. **Si A es simétrica, las matrices A_k convergen a una matriz diagonal** con los eigenvalores en la diagonal

Ahora, les toca a ustedes implementar y poner a prueba el método QR para encontrar los eigenvalores de una matriz cuadrada A .

Ejercicio 1: Cálculos Con el Polinomio Característico

Por medio del método del polinomio característico, calculen los eigenvalores de la siguiente matriz

$$A = \begin{pmatrix} 5 & -2 \\ -2 & 8 \end{pmatrix}$$

Solución:

Ejercicio 2: El Método QR Simple

En este ejercicio programarán el método QR para calcular los eigenvalores de una matriz real A de tamaño $n \times n$. Comenzaremos suponiendo que la matriz A es simétrica.

Paso 1:

En la siguiente celda, copia y pega la función que programaste en la práctica pasada para encontrar la descomposición QR de una matriz A .

```
import numpy as np
```

Solución:

Paso 2:

Aquí comenzamos a implementar el método QR . En este primer paso implementarán el método QR Simple que se encuentra en el Algoritmo 1 al principio de la práctica. Deberán programar una función que reciba como parámetros una matriz real A que sea cuadrada y

simétrica y el número de iteraciones N . El método deberá regresar la matriz A_k después de las N iteraciones.

Una vez que hayas programado la función, **pruebala con la matriz A del ejercicio 1 y con 10 iteraciones**, extrae los valores de la diagonal de la matriz resultante y compáralos con los eigenvalores calculados en el Ejercicio 1.

Definimos la matriz A para ti en la siguiente celda:

```
A = np.array([[5, -2], [-2, 8]])
```

Se puede usar

np.diag(v, k=0): Extrae la diagonal o construye una matriz diagonal. Donde v es arreglo o una matriz y k es una opción para elegir la k -diagonal, por defecto $k = 0$ se refiere a la diagonal principal.

```
print(A)
np.diag(A)
```

```
[[ 5 -2]
 [-2  8]]
```

```
array([5, 8])
```

```
np.diag(np.array([1, 2, 3, 4]), k = -1)
```

```
array([[0, 0, 0, 0, 0],
```

En esta celda implementa tu función:

Solución:

```
[1, 0, 0, 0, 0],
[0, 2, 0, 0, 0],
[0, 0, 3, 0, 0],
[0, 0, 0, 4, 0]])
```

Ejercicio 3: El Método QR

Hasta ahora nuestro método QR simple ha funcionado bien, pero queremos poder controlar la precisión del mismo. Para ello hemos de agregar una restricción que nos asegure que alcancemos la precisión deseada. En este caso pediremos que **los valores fuera de la diagonal** de nuestra matriz A_k sean todos **más chicos que un ε dado**.

Teniendo esto en mente, programa una función para el método QR que reciba como parámetros una matriz real A **que sea cuadrada y simétrica**, ε y un número máximo de iteraciones N en caso de que no se alcance la precisión deseada, el valor de la precisión deseada. El método deberá regresar la matriz A_k una vez que esta tenga valores fuera de la diagonal menores que ε o que se haya alcanzado el número máximo de iteraciones.

Una vez que hayas programado la función, pruebala con **la matriz A del ejercicio 1, con tolerancia $\varepsilon = 1 \times 10^{-10}$ y número máximo de iteraciones 1000**; extrae los valores de la diagonal de la matriz resultante y compáralos con los eigenvalores calculados en el Ejercicio 1

Puede usar

```
a = np.arange(4, 20).reshape(4, 4)
a
array([[ 4,  5,  6,  7],
```

np.triu_indices(n, k, m = n): Método que devuelve los índices que corresponden a una matriz triangular superior de una matriz de una matriz de dimensión $n \times m$. Donde **n** es la dimensión de la matriz (por defecto $n = m$) y **k** es una opción para elegir la k -diagonal.